

PROJETO 1

1. REQUISITOS DO SISTEMA

Deve-se desenvolver um sistema web que será um **portfólio web de receitas culinárias** cadastradas pelos alunos. Devem poder acessar esse portfólio os **alunos**, um **administrador do sistema** e qualquer **usuário externo**.

1.1 REQUISITOS DO USUÁRIO ALUNO

1.1 – Fazer login

O aluno não poderá se autocadastrar no sistema, mas deverá poder fazer login.

1.2 – Cadastrar receitas

Os alunos deverão poder cadastrar todas as suas receitas, que devem ser vinculadas a **categorias** previamente cadastradas no sistema.

Uma receita pode estar vinculada a quantas categorias o aluno desejar.

Deve existir um relacionamento de **muitos para muitos (N:N)** entre a tabela de **receitas** e a tabela de **categorias**.

Cada receita deve ter:

- nome
- descrição
- link externo

1.3 – Cada receita pode ser desenvolvida por mais de um aluno

Nesse caso, um único aluno poderá cadastrar a receita, mas todos os alunos vinculados a ela se tornam responsáveis e podem editar todos os seus dados, inclusive trocar ou remover categorias e/ou alunos cadastrados como responsáveis.

1.4 – Cada aluno pode cadastrar suas habilidades culinárias e definir seu nível em cada uma

O nível deve ser definido em uma escala de **0 a 10**.

Todas as habilidades devem estar previamente cadastradas no sistema.

Um aluno pode ter zero ou mais habilidades.

1.5 – Cada aluno pode editar e/ou excluir suas receitas, as categorias de cada receita e suas próprias habilidades

1.2 REQUISITOS DO USUÁRIO ADMINISTRADOR

1.6 – O administrador deve poder, quando logado no sistema, cadastrar, listar, editar e excluir alunos, categorias e habilidades no sistema.

1.3 REQUISITOS DO USUÁRIO EXTERNO

1.7 – O usuário externo deve poder, sem estar logado no sistema, visualizar todas as receitas cadastradas pelos alunos.

1.8 – O usuário externo deve poder, sem estar logado no sistema, visualizar todas as receitas vinculadas a uma categoria que ele tenha escolhido entre as categorias já cadastradas no sistema.

1.9 – O usuário externo deve poder visualizar um relatório com a proporção de alunos que dominam cada uma das habilidades cadastradas no sistema.

2. CONFIGURAÇÃO DOS GRUPOS

O projeto pode ser desenvolvido de forma individual, em duplas ou em grupos de, no máximo, três pessoas.

3. TECNOLOGIAS QUE DEVEM SER UTILIZADAS

O sistema deve ser desenvolvido com as tecnologias apresentadas na disciplina para o desenvolvimento de uma aplicação web seguindo o estilo arquitetural **MVC**.

Caso o aluno queira utilizar uma tecnologia diferente, deve:

- consultar a professora e obter aprovação;
- desenvolver o trabalho sozinho.

4. ARTEFATOS QUE DEVEM SER ENTREGUES

Deve-se entregar todo o projeto do software, com exceção da pasta `node_modules`, compactado em um único arquivo `.rar`.

5. PROCESSO DE AVALIAÇÃO

A nota será atribuída de acordo com a porcentagem de funcionalidades desenvolvidas e apresentadas (**50%**) ao professor em uma prova de autoria (**50%**), que deve ser realizada presencialmente.

Se o aluno apenas enviar o projeto e não fizer a apresentação, a nota atribuída será zero.

6. DICA EXTRA PARA O DESENVOLVIMENTO

Uma boa forma de iniciar o desenvolvimento do sistema é implementar o projeto **em etapas**, seguindo a ordem abaixo:

1. Modelagem do banco de dados

Comece criando as tabelas principais do sistema, como:

- alunos
- receitas
- categorias
- habilidades

Em seguida, implemente as tabelas de relacionamento necessárias para representar os vínculos **muitos para muitos (N:N)**.

2. Estrutura do projeto em MVC

Organize o projeto utilizando o padrão **Model-View-Controller**, separando:

- **Models** → responsáveis pela comunicação com o banco de dados
- **Controllers** → responsáveis pela lógica da aplicação
- **Views** → responsáveis pela interface com o usuário

3. Implementação por funcionalidades

Desenvolva o sistema em pequenos blocos, por exemplo:

- primeiro o **login**
- depois o **CRUD de receitas**
- depois o **CRUD de categorias**
- depois o **cadastro de habilidades**
- por fim as **páginas públicas e relatórios**

4. Testes constantes

Teste cada funcionalidade logo após implementá-la. Isso facilita a identificação de erros e evita problemas maiores no final do projeto.

J **Sugestão:** utilizar ferramentas como **Postman** ou **Insomnia** para testar as rotas da aplicação durante o desenvolvimento.